Suport laborator algoritmica grafurilor

II. Algoritmi fundamentali pe grafuri

II.1. Algoritmul lui Moore pentru graf orientat neponderat

Pentru un graf orientat neponderat $G = (V, E)$ si un varf sursa s, algoritmul lui Moore determina lantul minim, in numar de muchii, de la s la toate varfurile accesibile din s. Algoritmul este bazat pe parcurgerea in latime (Breadth-First Search).

Se folosesc urmatoarele structuri:

$$l(v) = \text{lungimea lantului minim de la } s \text{ la } v$$

$$p(v) = \text{parintele lui } v \text{ in arborele BFS}$$

Moore(G, s)

1. Pentru fiecare $v \in V$ executa
2. $\quad l(v) \leftarrow \infty$
3. $\quad p(v) \leftarrow \text{NIL}$
4. $l(s) \leftarrow 0$
5. $Q \leftarrow \emptyset$
6. Adauga s in Q
7. Cat timp $Q \neq \emptyset$ executa
8. $\quad$ Extrage x din Q
9. $\quad$ Pentru fiecare $y \in Adj(x)$ executa
10. $\quad\quad$ Daca $l(y) = \infty$ atunci
11. $\quad\quad\quad l(y) \leftarrow l(x) + 1$
12. $\quad\quad\quad p(y) \leftarrow x$
13. $\quad\quad\quad$ Adauga y in Q
14. Returneaza l si p

Problema 1. Implementati algoritmul lui Moore pentru un graf orientat neponderat. Datele sunt citite din fisierul `graf.txt`. Primul rand din fisier contine numarul varfurilor, iar urmatoarele randuri contin muchiile grafului. Programul trebuie sa afiseze lantul cel mai scurt dintr-un varf sursa, citit de la tastatura.

Rezolvare

Fisierul `graf.txt` descrie un graf orientat neponderat. Deoarece muchiile nu au costuri, lungimea unui drum se masoara prin numarul de muchii parcurse. In aceasta situatie, algoritmul potrivit este BFS, adica algoritmul lui Moore.

Pasii de rezolvare sunt:

1. se citeste numarul de varfuri;
2. se construieste lista de adiacenta;
3. se citeste varful sursa;
4. se ruleaza algoritmul lui Moore;
5. pentru un varf destinatie dat, lantul minim se reconstruieste folosind vectorul parinti p.

Daca dorim sa afisam lantul minim de la s la un varf t, pornim din t si urmarim succesiv:

$$t,\ p(t),\ p(p(t)),\ \ldots$$

pana ajungem la s, dupa care inversam ordinea obtinuta.

Notatii:

- daca un varf v ramane cu $l(v) = \infty$, atunci acesta nu este accesibil din sursa;
- arborele definit de vectorul p este arborele descoperit de BFS.

II.2. Inchiderea tranzitiva a unui graf orientat

Inchiderea tranzitiva a unui graf orientat descrie relatia de accesibilitate intre varfuri. Pentru un graf cu n varfuri, se defineste matricea

$$T = (t_{ij})_{i,j=1,n}$$

unde

$$t_{ij} = \begin{cases} 1, & \text{daca exista drum de la } i \text{ la } j, \\ 0, & \text{altfel.} \end{cases}$$

Pentru determinarea inchiderii tranzitive vom folosi algoritmul Floyd-Warshall adaptat pentru accesibilitate. Ideea este sa permitem, pe rand, fiecare varf k ca varf intermediar intre i si j.

Floyd-Warshall pentru accesibilitate

1. Pentru $i = 1, n$ executa
2. Pentru $j = 1, n$ executa
3.   Daca exista muchie $i \to j$ atunci $T[i][j] \leftarrow 1$
4.   Altfel $T[i][j] \leftarrow 0$
5. $T[i][i] \leftarrow 1$
6. Pentru $k = 1, n$ executa
7. Pentru $i = 1, n$ executa
8.   Pentru $j = 1, n$ executa

9. $T[i][j] \leftarrow T[i][j] \vee (T[i][k] \wedge T[k][j])$

10. Returneaza T

Problema 2. Sa se determine inchiderea tranzitiva a unui graf orientat. Inchiderea tranzitiva poate fi reprezentata ca matricea care descrie, pentru fiecare varf in parte, care sunt varfurile accesibile din acest varf.

Rezolvare

Problema cere sa determinam, pentru fiecare pereche de varfuri (i, j), daca exista sau nu un drum de la i la j. Rezultatul final este o matrice binara.

Se porneste de la matricea de adiacenta logica a grafului:

$$T[i][j] = 1 \iff \text{exista muchie directa } i \to j$$

iar pe diagonala principala se pune valoarea 1, deoarece orice varf este accesibil din el insusi printr-un drum de lungime 0.

Apoi, pentru fiecare varf intermediar k, testam daca un drum de la i la j poate fi obtinut prin compunerea a doua drumuri:

$$i \rightsquigarrow k \quad \text{si} \quad k \rightsquigarrow j.$$

Daca ambele exista, atunci si drumul de la i la j exista.

Formula de actualizare este:

$$T[i][j] \leftarrow T[i][j] \vee (T[i][k] \wedge T[k][j]).$$

La finalul algoritmului, matricea T este chiar matricea inchiderii tranzitive.

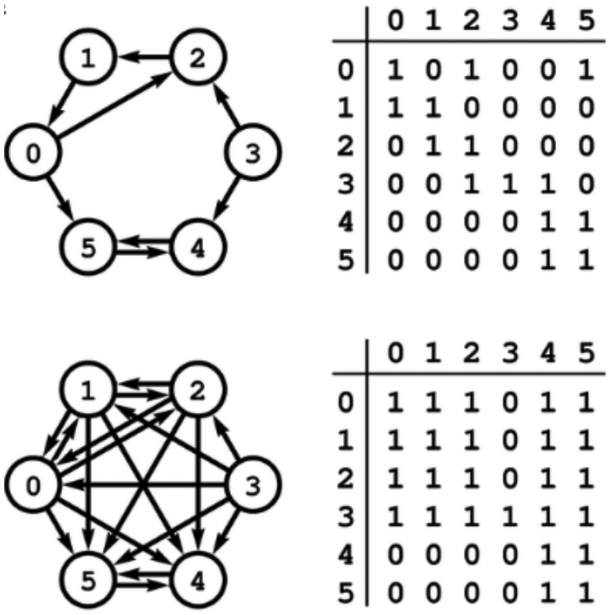

Figure 1: Exemplu de inchidere tranzitiva

In figura de mai sus, pentru graful initial se construieste matricea care indica toate relatiile de accesibilitate. Astfel, pe linia i apar toate varfurile in care se poate ajunge pornind din varful i.

II.3. Rezolvarea unui labirint 2D

Un labirint bidimensional poate fi privit ca o matrice. Celulele libere reprezinta pozitii in care ne putem deplasa, iar peretii reprezinta pozitii interzise. Pentru rezolvarea problemei se foloseste algoritmul lui Lee, care este o aplicatie a parcurgerii BFS pe o matrice.

Consideram:

- `S` – celula de start;
- `F` – celula finala;
- `1` sau o alta valoare speciala – obstacol;
- celulele libere – pozitii accesibile.

Deplasarea se face in patru directii, definite prin vectorii:

$$dx = (-1,\ 1,\ 0,\ 0), \qquad dy = (0,\ 0,\ -1,\ 1).$$

Lee(M, S)

1. Pentru fiecare celula (i, j) executa
2. $\quad dist[i][j] \leftarrow -1$
3. $Q \leftarrow \emptyset$
4. $dist[S] \leftarrow 0$
5. Adauga S in Q
6. Cat timp $Q \neq \emptyset$ executa
7. $\quad$ Extrage (x, y) din Q
8. $\quad$ Pentru $k = 0, 3$ executa
9. $\quad\quad nx \leftarrow x + dx[k]$
10. $\quad\quad ny \leftarrow y + dy[k]$
11. $\quad\quad$ Daca (nx, ny) este valid, liber si $dist[nx][ny] = -1$ atunci
12. $\quad\quad\quad dist[nx][ny] \leftarrow dist[x][y] + 1$
13. $\quad\quad\quad$ Adauga (nx, ny) in Q
14. Returneaza $dist$

Problema 3. Sa se scrie un program care gaseste o solutie pentru unul din urmatoarele labirinturi: `labirint_1.txt`, `labirint_2.txt`, `labirint_cuvinte.txt`. Pentru labirintul 1 si 2 punctul de pornire este indicat de litera `S` si punctul de oprire este indicat de litera `F`. Pentru labirintul cu cuvinte nu exista o solutie unica.

Rezolvare

Modelam labirintul ca o matrice. Fiecare celula libera este un nod, iar dintr-o celula ne putem muta doar in vecinii sai directi: sus, jos, stanga, dreapta. Deoarece toate mutarile au acelasi cost, problema gasirii unui drum minim devine o problema BFS pe matrice.

Algoritmul lui Lee marcheaza distanta minima de la pozitia de start la fiecare celula accesibila. Astfel:

- initial, toate distantele sunt necunoscute;
- pozitia `S` primeste valoarea 0;
- se exploreaza succesiv vecinii valizi ai fiecarei celule extrase din coada;
- fiecare vecin liber si nevizitat primeste distanta curenta plus 1.

Daca la final celula `F` are valoare diferita de -1, atunci exista solutie. Drumul se poate reconstrui pornind din `F` si alegand la fiecare pas un vecin cu valoare cu 1 mai mica, pana se ajunge la `S`.

Pentru labirintul cu cuvinte pot exista mai multe drumuri valide, iar algoritmul returneaza unul dintre drumurile minime, in functie de ordinea in care sunt analizate directiile de deplasare.

II.4. Arborele descoperit de algoritmul BFS

Aplicarea algoritmului BFS dintr-un varf sursa s produce un arbore de descoperire. Acesta este format din muchiile

$$(p(v), v), \quad \text{pentru toate varfurile } v \text{ accesibile din } s,\ v \neq s.$$

Problema 4. Pentru un graf dat sa se afiseze pe ecran varfurile descoperite de algoritmul BFS si distanta fata de varful sursa.

Rezolvare

Problema se rezolva direct prin aplicarea algoritmului Moore. In timpul executiei, fiecare varf este descoperit pentru prima data atunci cand distanta sa devine finita. In acel moment:

- se cunoaste distanta minima fata de sursa;
- se cunoaste parintele prin care a fost descoperit.

Prin urmare, pentru fiecare varf v se pot afisa:

$$v, \quad l(v), \quad p(v).$$

Daca dorim ordinea descoperirii, este suficient sa afisam varful in momentul in care acesta este introdus pentru prima data in coada.

Arborele BFS este util deoarece arata toate drumurile minime de la sursa catre varfurile accesibile.

II.5. Padurea descoperita de DFS

Parcurgerea in adancime (Depth-First Search) exploreaza graful urmand cat mai adanc fiecare ramura inainte de a reveni. Daca graful nu este conex, atunci DFS produce o padure de arbori.

DFS(G)

1. Pentru fiecare $v \in V$ executa
2. $\quad vizitat[v] \leftarrow false$
3. Pentru fiecare $v \in V$ executa
4. $\quad$ Daca $vizitat[v] = false$ atunci
5. $\quad\quad$ `dfs_visit`(G, v)

dfs_visit(G, v)

1. $vizitat[v] \leftarrow true$
2. Afiseaza v
3. Pentru fiecare $u \in Adj(v)$ executa
4. $\quad$ Daca $vizitat[u] = false$ atunci
5. $\quad\quad$ `dfs_visit`(G, u)

Problema 5. Pentru un graf dat sa se afiseze pe ecran varfurile descoperite de apelul recursiv al procedurii `dfs_visit(graf, nod)`.

Rezolvare

Se initializeaza toate varfurile ca nevizitate, dupa care se parcurg toate varfurile grafului. De fiecare data cand se gaseste un varf nevizitat, se porneste un nou apel `dfs_visit`. Acest apel construieste un arbore din padurea DFS.

Ordinea in care sunt afisate varfurile depinde de:

- ordinea varfurilor din structura principala;
- ordinea vecinilor din listele de adiacenta.

Daca graful este conex, atunci rezultatul este un singur arbore DFS. Daca graful are mai multe componente, atunci fiecare componenta va genera un arbore separat, iar ansamblul lor formeaza padurea DFS.